
More Batches is All You Need

Princeton Liu
206267879

Richemond Shin
606257232

Abstract

Speech brain-computer interfaces (BCI) are capable of allowing those with neurological conditions to communicate. This project investigates decoding temporal neural spike train activity into phoneme sequences with the aim of predicting text given intracranial neural recordings from a participant with Amyotrophic Lateral Sclerosis (ALS). We begin with a unidirectional Gated Recurrent Unit (GRU) model trained with Connectionist Temporal Classification (CTC) loss and use it as the fixed baseline, exploring architectural modifications including residual attention layers, dropout tuning, and feed-forward blocks to improve model performance. The baseline model reached a Phoneme Error Rate (PER) of 0.223, while our best modified model achieved 0.167, indicating modest but meaningful improvement by decreasing error rate by 25 percent. Due to the increased model complexity, the modified model required more training batches to converge. Results suggest the decoding task is sensitive to temporal alignment, and gains from additional model complexity depend on regularization and optimization stability.

1 Introduction

Neurological diseases such as ALS can lead to paralysis, hampering the afflicted's communication ability. Fortunately, speech BCI techniques which decode speech directly from neural activity have made significant strides in recent years, reaching the ability to decode large volumes of neural data with high accuracy [1]. Neural spike trains evolve over time and must be mapped into discrete linguistic units, making sequence modeling a natural approach. Recurrent architectures such as GRUs are widely used due to their ability to retain temporal memory without requiring explicit alignment.

We use the Brain-to-Text Benchmark '24 dataset, which contains intracranial microelectrode array recordings from a participant with ALS performing attempted speech [2]. The dataset provides high-resolution neural activity aligned to sentence-level prompts, along with predefined training, validation, and test splits. Because no ground-truth phoneme-to-time alignment is available, models must infer both content and timing from the input signal. CTC loss is therefore a central component, as it enables supervision when the input and output sequences differ in length and lack explicit alignment [3].

A baseline Gated Recurrent Unit (GRU) model accompanies the dataset and serves as the reference system for all benchmark comparisons [4]. It incorporates day-specific input transformations, multi-layer recurrent processing, and stride-based output generation.

Although functional, we expect it to exhibit some limitations since it does not take advantage of techniques such as regularization, attention mechanisms, or other architectural modifications, allowing us to explore those domains. As a result, our goal for this project is to improve the PER of a GRU-based neural signal decoder by modifying the training process or the model's architecture.

2 Methods

2.1 Dataset

Our project utilizes the publicly released Brain-to-Text Benchmark '24 dataset, which contains intracortical microelectrode array (MEA) recordings from a single participant diagnosed with ALS. Neural activity was collected while the participant attempted to speak prompted sentences. Two MEAs were implanted in the ventral premotor cortex (area 6v), one ventral and one dorsal, each containing 64 electrodes. For each channel, both spike-band power and threshold-crossing signals were extracted, yielding a total of 256 features per time bin. Neural recordings were segmented into 20 ms time bins (50 Hz resolution) and z -scored within blocks of 20–50 sentences to normalize activity across sessions [2, 4, 5].

The dataset comprises a total of 10,880 sentence trials. In each trial, the participant was shown a sentence and cued to speak; only neural activity recorded during the “go” phase, when speech was attempted, is used for decoding. The benchmark provides predefined train, validation, and test splits. Training data includes 8,800 sentences, validation contains 880, and the test set contains 1,200 sentences. Training and validation sentences were collected over 24 days spanning approximately four months, while the test set draws from 15 of those 24 days [4, 5].

2.2 Evaluation

Primary metric was phoneme error rate (PER) computed on test sequences. PER is defined as

$$\text{PER} = \frac{S + D + I}{N},$$

where S is the number of substitutions, D the number of deletions, I the number of insertions, and N the total number of phonemes in the reference sequence[5].

Since our models produce phoneme logits over time using a CTC decoder, PER directly quantifies how accurately the predicted sequences match the reference phonemes. During training, we monitored both training and testing loss curves to track convergence and overfitting.

2.3 Baseline Model

The baseline architecture was a GRU decoder trained with CTC-loss [4]. Inputs were projected into a hidden representation, processed through stacked GRU layers, and mapped to phoneme logits over time (see Fig. 1).

Roughly speaking, the reason why GRUs are well-suited and chosen for our neural decoding task is because of its stateful architecture, which helps with processing data where sequence has a meaningful role. In our case, neural data is collected over time, which is the index for our sequence. To demonstrate how the GRU computation works, let us define:

$$\begin{aligned} x_t &:= \text{input at time } t \text{ (spike train data at a singular time step)} \\ h_t &:= \text{GRU hidden state at time } t \end{aligned}$$

In simplest terms, the GRU is a function which computes h_t (the new hidden state) in terms of x_t (the current input) and h_{t-1} (the previous hidden state). There are 2 main components involved with this computation, namely the “update gate” z_t and the “reset gate” r_t , which are both defined as:

$$\begin{aligned} z_t &:= \sigma(W_z x_t + U_z h_{t-1}) \\ r_t &:= \sigma(W_r x_t + U_r h_{t-1}) \end{aligned}$$

Note that these two “gates” contain learnable parameters W_z, U_z, W_r, U_r , which are optimized for during training. Also note that the “sigmoid” activation function σ is defined as:

$$\sigma(x) := \frac{1}{1 + e^{-x}}$$

After computing z_t and r_t , the GRU computes a “candidate hidden state” $\tilde{h}_t$ defined as:

$$\tilde{h}_t := \tanh(W_h x_t + U_h (r_t \odot h_{t-1}))$$

Finally, the new hidden state is computed by combining the previous and candidate hidden states with a ratio of $1 - z_t$ and z_t respectively:

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

After computing the new hidden state, the GRU makes a phoneme prediction by decoding the hidden state using a fully connected linear layer, where the output is logit probabilities over each of the 39 phonemes (including the "silence token") [4].

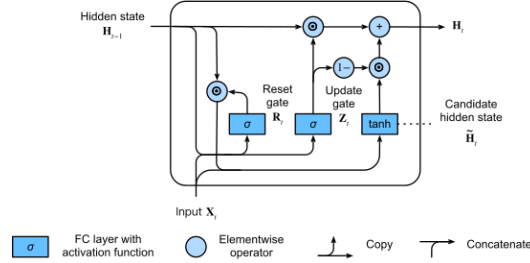


Figure 1: Visualization of a single-layered GRU model architecture [6].

Mathematically, the input to the GRU at time step t can be expressed as a concatenation of multiple time steps according to the kernel length k and stride length s . Let $x_{t:t+k-1}$ denote the sequence of inputs from time t to $t + k - 1$. Then the GRU input at "step" i after applying the stride is:

$$\tilde{x}_i := [x_{i \cdot s}, x_{i \cdot s + 1}, \dots, x_{i \cdot s + k - 1}] \in \mathbb{R}^{k \cdot n_{\text{features}}}$$

where n_{features} is the number of recorded neurons (256 in our case). This vector $\tilde{x}_i$ is fed into the GRU to compute the hidden state:

$$h_i = \text{GRU}(\tilde{x}_i, h_{i-1})$$

Effectively, this allows the GRU to capture correlations across k contiguous time steps, while moving forward in time with stride s , giving a controllable local temporal context without increasing the sequence length excessively.

For multiple layers, the output hidden state of layer l at step i , $h_i^{(l)}$, is used as the input to the next layer $l + 1$ at the same step:

$$h_i^{(l+1)} = \text{GRU}^{(l+1)}(h_i^{(l)}, h_{i-1}^{(l+1)})$$

This stacked architecture enables the model to capture hierarchical temporal features, where lower layers focus on short-term patterns and higher layers can represent more abstract temporal dependencies.

2.4 Modified Models

To improve upon the baseline model, we experimented with many different architectural and training modifications. Each modification targets a specific limitation observed in the original model, and we go over the intuition behind why each modification improves performance.

2.4.1 Residual self-attention applied over GRU outputs

We first notice that the hidden state of the GRU is highly local, meaning that the hidden state must encapsulate the essence of all data from previous time steps in a single vector. This is limited in the fact that phonemes are highly dynamic, meaning that a specific phoneme at time t could alter the way the phoneme at a previous time step should be interpreted. To remedy this limitation, we introduce "self-attention," which is a mechanism which allows our model to update previous time steps based on how they "attend" to the current one (see Fig. 2).

We first define the hidden state matrix $H \in \mathbb{R}^{T \times d}$ which contains the hidden states of all time steps. If future hidden states have not been computed, the columns will have all zero entries. Then we compute "query", "key", and "value", matrices $Q, K, V \in \mathbb{R}^{T \times d}$ using linear transformations:

$$Q := HW_Q \quad K := HW_K \quad V := HW_V$$

Note that $W_Q, W_K, W_V \in \mathbb{R}^{d \times d}$ are all learnable parameters which are optimized for during training. Then for each timestep t , we compare the similarity between its query with the keys of all other time steps by doing an inner product:

$$\text{score}(t, t') := \frac{\langle q_t, k_{t'} \rangle}{\sqrt{d}} \quad \text{scores in matrix form: } S := \frac{QK^T}{\sqrt{d}}$$

Finally, we have a $T \times T$ matrix S where the entry at i, j represents how the hidden state at time step i attends to the hidden state at time step j . We finish computing for attention by performing a softmax over S (to regularize attention scores) then weighting the attention scores based on the "value" matrix V , ending with:

$$\text{SelfAttention}(H) = \text{softmax}(S) \cdot V$$

Below is some python-like pseudocode which shows roughly how attention is computed [7]. Note that we must "mask" all future hidden states to prevent attending to states which have not yet been computed. Also note that layer normalization and dropout are computed on hidden state and attention weights in order to prevent activation explosion and prevent dependence on specific time steps (further explanations in section 2.4.5).

Listing 1: Residual Multi-Head Attention Layer

```

1 def add_attention(hid):
2     hid_norm = layer_norm(hid)
3     Q = q_proj(hid_norm)
4     K = k_proj(hid_norm)
5     V = v_proj(hid_norm)
6
7     scores = (Q @ K.T) / (d ** 0.5)
8     scores = scores.masked_fill(~attn_mask, -1e9)
9
10    weights = torch.softmax(scores)
11    weights = dropout(weights)
12
13    attn_out = weights @ V
14
15    return hid + attn_out

```

To capture multiple temporal dependencies simultaneously, we use "multi-headed attention". For h heads, separate linear projections are computed:

$$Q_i = HW_Q^{(i)}, \quad K_i = HW_K^{(i)}, \quad V_i = HW_V^{(i)}, \quad i = 1, \dots, h$$

Each head produces an attention output:

$$\text{head}_i = \text{Softmax}\left(\frac{Q_i K_i^T}{\sqrt{d_k}}\right) V_i, \quad d_k = d/h$$

The outputs are concatenated and projected:

$$\text{MultiHead}(H) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W_O$$

A residual connection and layer normalization yield the final hidden states:

$$H_{\text{attn}} = \text{LayerNorm}(H + \text{MultiHead}(H))$$

This allows some heads to focus on short-range interactions and others on long-range dependencies, while stabilizing training and preserving the original information.

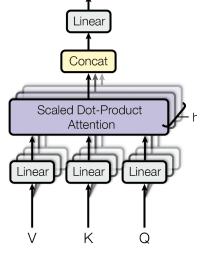


Figure 2: Visualization of a multi-headed attention layer [6].

2.4.2 Learning rate warm-up schedules

The learning rate controls the magnitude of each weight update during optimization. The baseline model used a fixed learning rate of 0.02, which enabled steady initial convergence but eventually led to a plateau in test CTC loss, as the model could no longer make sufficiently small adjustments to explore narrow regions of the loss landscape. To address this limitation, we introduced an exponential decay schedule that progressively reduces the learning rate over time, allowing finer-grained updates in later stages of training and enabling the model to reach lower-loss regions previously inaccessible.

We additionally applied a warm-up schedule at the start of training, gradually increasing the learning rate from a small initial value to the target initial learning rate over w batches. This guides early optimization toward a stable direction before decay begins, improving training stability.

Finally, we employed early stopping, halting training when the test loss failed to improve over a specified patience window. This prevents unnecessary computation and mitigates overfitting once the model has converged.

Thus, the actual learning rate at batch b with initial value l and exponential decay rate r after warm-up is then:

$$\text{LearningRate}(l, b, w) = \begin{cases} l \cdot \frac{b}{w} & \text{if } b \leq w \text{ (warm-up),} \\ l \cdot r^{b-w} & \text{otherwise (decay).} \end{cases}$$

2.4.3 Gradient clipping

During neural network training, excessively large gradients can lead to unstable updates. In sequence models such as the GRUs, gradients can sometimes grow exponentially during backpropagation through time, a phenomenon known as the exploding gradient problem. This can cause the model’s parameters to diverge, resulting in failed training or numerical instability [8].

The solution is to implement gradient clipping, a process which rescales gradients whenever their norm exceeds a predefined threshold. Suppose g is the vector of all gradients for a model’s parameters and τ is the clipping threshold. Our gradients are rescaled as:

$$g_{\text{clipped}} = \begin{cases} \tau \cdot \frac{g}{\|g\|} & \text{if } \|g\| > \tau \\ g & \text{otherwise} \end{cases}$$

So small gradients remain as is while large ones are changed to be within a stable range.

2.4.4 Mask-based data augmentation

When training sequence models on neural spike trains, models can overfit to specific temporal patterns in the training data. If a model memorizes exact sequences rather than learning robust temporal dependencies, it may perform poorly on unseen trials or noisy data. For instance, neural recordings in real-world scenarios often contain corrupted or missing segments due to electrode failures, motion artifacts, signal dropouts, or transient noise bursts. So sequence models trained solely on perfect, uninterrupted data can become brittle when exposed to real world data.

To mitigate this, we employ mask-based data augmentation, specifically time masking. During training, contiguous intervals of the input neural sequence are randomly masked by setting their values to zero. Formally, for a sequence $X = [x_1, x_2, x_3, \dots, x_T]$ a random interval $[x_s, x_e] \in X$ is selected and features in this interval are masked:

$$x_t = 0 \quad \text{for } t_s \leq t \leq t_e$$

By obscuring portions of the input, the model is forced to rely on surrounding temporal context rather than memorizing exact patterns.

2.4.5 Dropout and Layer Normalization

Deep recurrent networks often face two major challenges during training. First, overfitting can occur when models memorize training sequences instead of learning generalizable temporal patterns, especially with limited neural data. Second, internal covariate shift arises when the distribution of layer inputs changes during training, which can lead to gradient instability, slow convergence, and difficulty in optimizing deep recurrent layers.

Dropout addresses overfitting by randomly setting a fraction of the activations to zero during training. This prevents co-adaptation of neurons and encourages the network to learn redundant, distributed representations. Formally, given a layer output h , dropout applies a binary mask $m \sim \text{Bernoulli}(p)$ and scales the remaining activations:

$$\tilde{h} := \frac{h \odot m}{1 - p}$$

where p is the dropout probability. On top of this, we also implemented layer normalization which addresses internal covariate shift by normalizing activations across the feature dimension for each time step:

$$\text{LayerNorm}(x) = \gamma \odot \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$$

where $\mu := \frac{1}{H} \sum_{i=1}^H x_i$ and $\sigma^2 := \frac{1}{H} \sum_{i=1}^H (x_i - \mu)^2$ are the mean and variance across H features, γ and β are learnable scale and shift parameters, and $\epsilon = 10^{-5}$ is our numerical stability constant.

All models in this project were trained with AdamW optimization, using batch sizes of 64 and early stopping with a patience of 50. The learning rate decayed from 10^{-3} to 10^{-6} using patience-based scheduling. Dropout and layer normalization together improved both the stability of training, enabled faster convergence, and the generalization of phoneme decoding.

2.4.6 CTC Smoothing

CTC optimizes the probability of all valid alignment paths between the input sequence and the target transcription. A side effect is that the model is driven toward high-entropy collapse, where it assigns near-perfect confidence to a small number of alignment paths and forces most other paths toward zero. In plain terms, it means that standard CTC loss creates an overconfident model with binary posteriors, resulting in overfitting.

We resolve this through CTC smoothing, introducing controlled uncertainty into the target distribution and reallocating the probability mass:

$$\text{smoothed target} = (1 - \epsilon) \times \text{true target} + \epsilon \times u$$

where ϵ is our smoothing coefficient and u is a uniform distribution over the output vocabulary.

3 Results

The baseline GRU reached a PER of 0.223 on the test set after 10,000 training batches (see Fig. 3), requiring approximately 30 minutes of compute on an NVIDIA A100 GPU. Incorporating the modifications described in Section 2.4, along with targeted hyperparameter tuning and increased model capacity, reduced the test PER to 0.167 (~ 25 percent relative improvement). However, the enhanced architecture converged more slowly due to its increased complexity: its best performance was achieved after 25,000 batches, requiring over 1.5 hours of training on the same hardware (see Fig. 4).

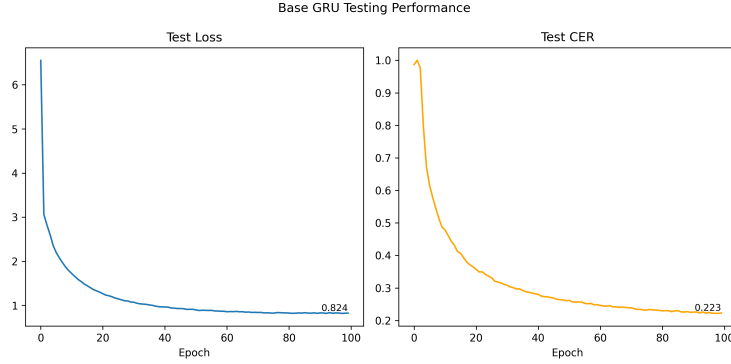


Figure 3: Visualization of Base GRU Testing Performance.

For the modified model, we found that the learning rate needed to be significantly lower than that of the baseline model (e.g., 5×10^{-4} vs. 0.02). This is consistent with the fact that attention layers are more sensitive to gradient noise, and large updates tend to destabilize training.

Additionally, because we applied substantial input augmentation, training loss initially remained higher than validation loss for roughly the first 4,500 batches. After this point, the training loss dropped well below the validation loss, suggesting a distribution mismatch between the augmented training data and the clean evaluation set.

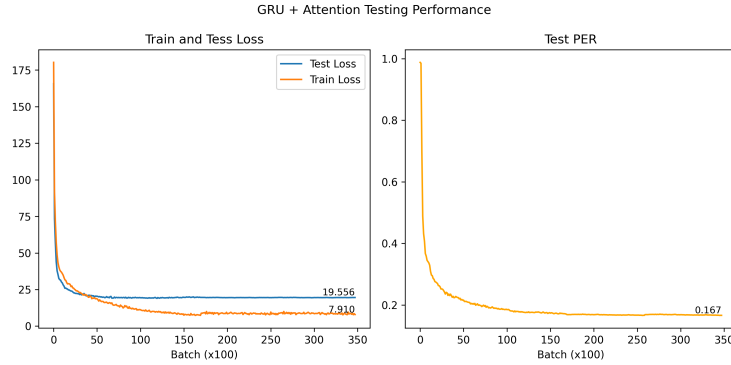


Figure 4: Visualization of Modified GRU Testing Performance.

4 Discussion

Results indicate that the dataset supports modest but not substantial gains from attention, suggesting that some degree of non-local temporal structure is present but not dominant. A more exhaustive hyperparameter search may be necessary to unlock the full benefits of attention, especially settings tuned for higher-capacity models.

Our modifications increased the model’s complexity, which necessitated longer training to reach convergence. The more expressive architecture required additional batches to fully optimize the parameters and stabilize learning.

An interesting observation arises when comparing test CTC loss and test PER: around 17,500 batches, the test CTC loss plateaus while PER continues to improve (Figure 4). This divergence suggests that the model is not improving likelihood calibration on unseen data, yet it is becoming increasingly consistent in its discrete phoneme predictions. One interpretation is that the model becomes more confident on its existing decision boundaries (lower PER), without learning better sequence-level probability structure (flat CTC loss). This raises a question about whether PER improvements at that stage reflect better generalization or simply sharper predictions.

References

- [1] Zengwei Yao, Wei Kang, Xiaoyu Yang, Fangjun Kuang, Liyong Guo, Han Zhu, Zengrui Jin, Zhaoqing Li, Long Lin, and Daniel Povey. CR-CTC: Consistency regularization on CTC for improved speech recognition. In *Proceedings of the International Conference on Learning Representations*, 2025. arXiv:2410.05101v4.
- [2] Francis R. Willett, Erica M. Kunz, Chaoyang Fan, and et al. Data for: A high-performance speech neuroprosthesis, 2023. Brain-to-Text Benchmark '24 dataset.
- [3] Awni Hannun. Sequence modeling with ctc. *Distill*, 2017. doi: 10.23915/distill.00008. Online; available at <https://distill.pub/2017/ctc>.
- [4] Francis R. Willett, Erica M. Kunz, Chaoyang Fan, and et al. A high-performance speech neuroprosthesis. *Nature*, 620:1031–1036, 2023. doi: 10.1038/s41586-023-06377-x.
- [5] Jonathan C. Kao. *Predicting Text from Intracranial Neural Signals: Project Instructions*. ECE C143A, University of California, Los Angeles, 2024. Unpublished class instructions. Available from course website.
- [6] Aston Zhang, Zachary C. Lipton, Mu Li, and Alexander J. Smola. *Dive into Deep Learning*, 2021. <https://d2l.ai>.
- [7] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Proceedings of the Neural Information Processing System*, 2017. arXiv:1706.03762v7.
- [8] Jingzhao Zhang, Tianxing He, Suvrit Sra, and Ali Jadbabaie. Why gradient clipping accelerates training: A theoretical justification for adaptivity. In *Proceedings of the International Conference on Learning Representations*, 2020. arXiv:1905.11881v2.

A Hyperparameters

We list the hyperparameters of the baseline and modified models in Table 1.

Table 1: Comparison of OLD (baseline) and NEW (modified) hyperparameters and results.

Hyperparameter / Metric	OLD (Baseline)	NEW (Modified)
Model Name	speechBaseline4	attentionLayerNorm6
Bidirectional	False	False
Sequence Length	150	150
Max Time Series Length	1200	1200
Number of Classes	40	40
Input Features	256	256
Stride Length	4	4
Kernel Length	32	32
Batch Size	64	64
Number of Batches	10000	30000
Number of Units	1024	1536
Number of Layers	5	7
Dropout	0.4	0.5
Attention Dropout	-	0.2
Learning Rate Start	0.02	5e-4
Learning Rate End	0.02	5e-10
Learning Decay Rate	-	0.99985
L2 Decay	1e-5	1e-4
White Noise SD	0.8	0.8
Constant Offset SD	0.2	0.2
Gaussian Smooth Width	2.0	2.0
Gradient Clipping	-	1.0
Warm-up Steps	-	1000
Early Stopping Patience	-	50
Max Mask Length	-	40
Number of Masks	-	4
Results		
Test CER	0.223	0.167
Training Time	32 min	61 min

B Code

The Jupyter Notebook is available on the next page.